

Artificial Intelligence(H) PJ1 实验报告

孔恩燊 23307130021 2026.4

Part 1

Part 1 要求使用 `Numpy` 自行实现反向传播，完成拟合正弦函数与图像分类两个任务。

1. 回归任务 (Regression)

1.1 任务与数据

任务为在区间 $x \in [-\pi, \pi]$ 上拟合 $y = \sin(x)$ ，于是我在区间内对 x 均匀采样 (`np.linspace`)，得到成对样本 $(x, \sin x)$ ，并将 x, y reshape 为列向量，形状为 $(N, 1)$ ，便于矩阵运算。

1.2 模型与损失

采用多层全连接网，隐层使用 `tanh` 激活，输出层为无激活的线性，用于回归。每层线性变换形式为 $z = aW + b$ ，其中 b 为偏置，对每一层引入与输出维度一致的偏置向量，并在前向中通过广播加到 batch 上。反向传播中偏置梯度为对 batch 维求和 (`axis=0, keepdims=True`)，与要求一致。

损失函数为均方误差 (MSE)：

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

对最后一层线性输出求 MSE 梯度后，按链式法则逐层回传，隐层处乘以 `tanh` 的导数。

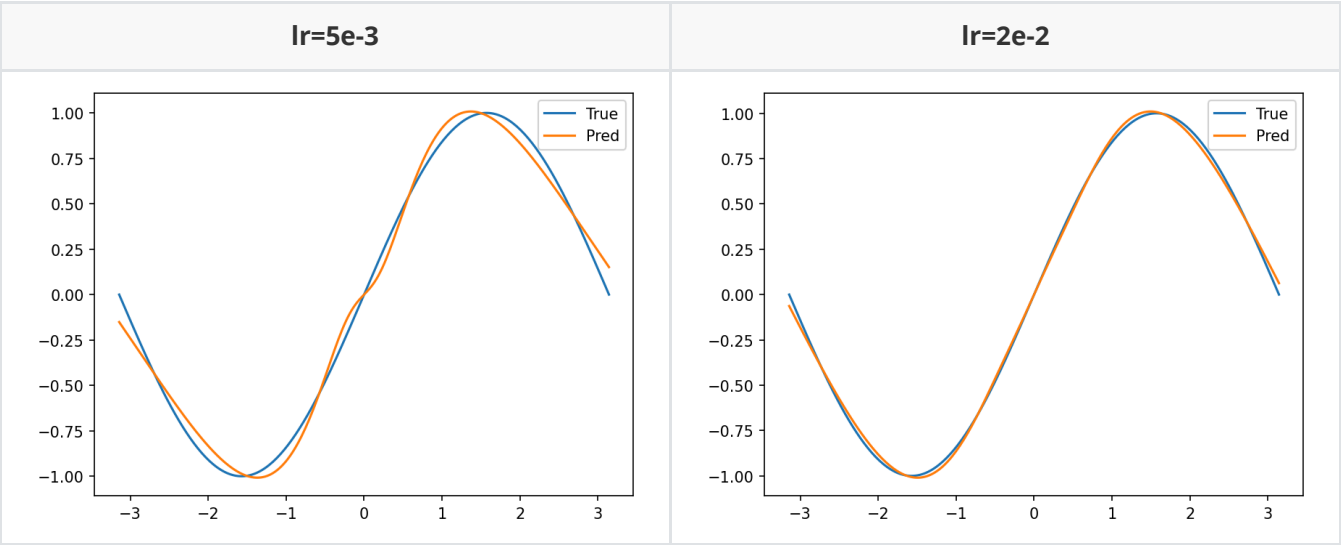
1.3 实现要点

我使用梯度下降法更新参数，权重采用常用随机初始化，偏置置零初始化。训练循环为：前向 $\rightarrow$ 计算 MSE $\rightarrow$ 反向 $\rightarrow$ SGD 更新。同一随机种子下结果可复现。代码主体在 `part1/regression.py` 中，各函数职责简述如下：

函数	作用
<code>set_seed</code>	固定 <code>numpy</code> 随机种子，便于复现初始化与数据顺序。
<code>generate_data</code>	在 $[-\pi, \pi]$ 上生成 $(x, \sin x)$ ，并 reshape 为 $(N, 1)$ 。
<code>init_params</code>	按 <code>layer_dims</code> 初始化各层 <code>w[i]</code> 、 <code>b[i]</code> ，其中 <code>w</code> 为随机初始化， <code>b</code> 为零初始化。
<code>forward</code>	前向：隐层 <code>tanh</code> ，最后一层线性输出，缓存 <code>z[i]</code> 、 <code>a[i]</code> 、 <code>y_pred</code> 。
<code>mse_loss</code>	计算均方误差。
<code>backward</code>	由 MSE 对输出层梯度出发，逐层回传。
<code>sgd_step</code>	按学习率对 <code>w[i]</code> 、 <code>b[i]</code> 做一步更新。
<code>train</code>	训练循环：前向—损失—反向—更新。
<code>predict</code>	给定 <code>x</code> 与训练好的参数，返回预测 $\hat{y}$ 。

函数	作用
<code>plot_fit</code>	绘制真值与预测曲线，并保存为 PNG。

最终回归拟合效果示例如下（均为 `layer_dims=[1,64,1]`，`seed=2026`，`epoch=20000`，仅学习率不同）：



1.4 对比实验

我尝试在固定训练轮数与随机种子的前提下，对比不同学习率与网络结构对收敛速度与最终 MSE 的影响。在实验中，固定配置为：

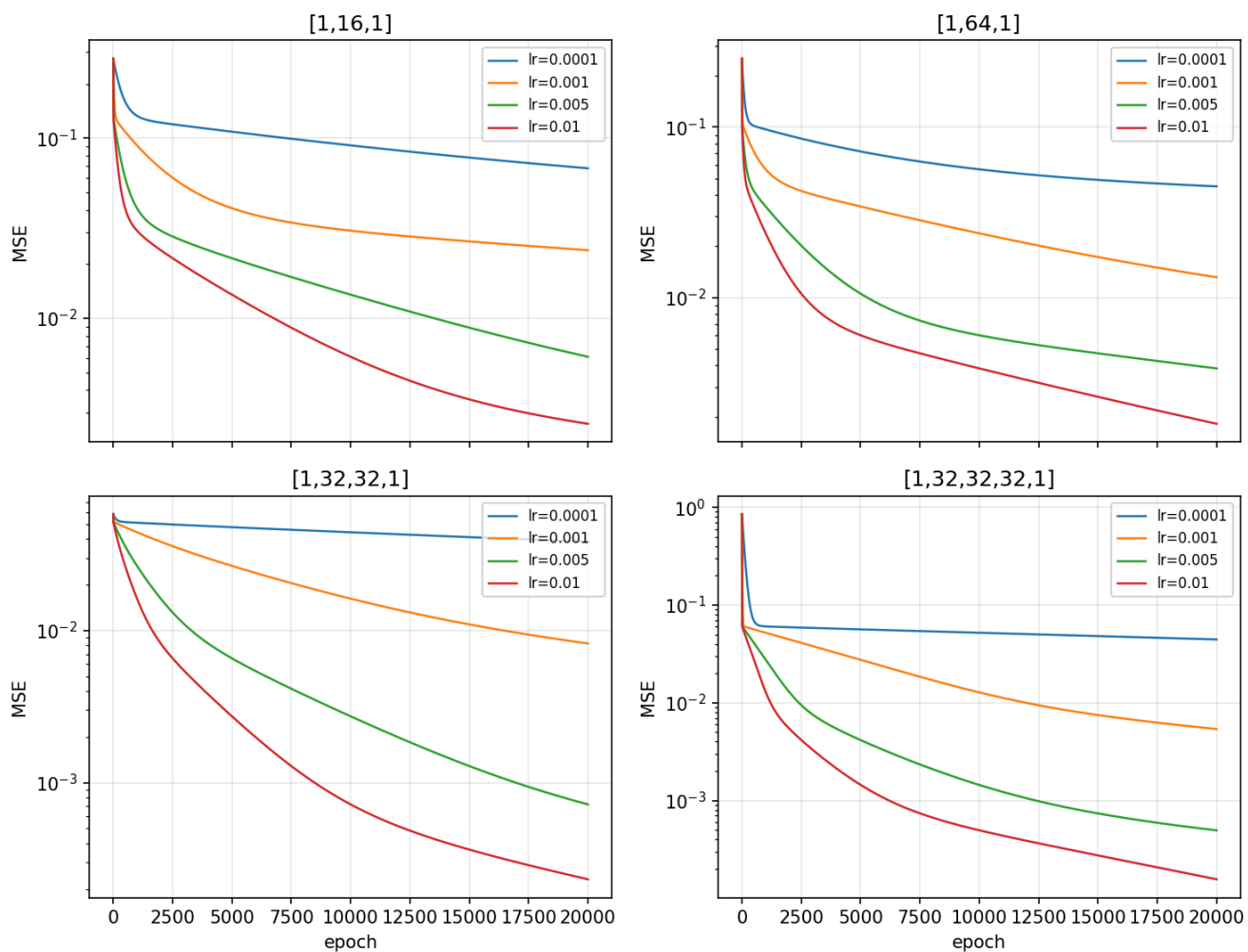
```
EPOCHS = 20000
SEED = 2026

# 5 个学习率
LEARNING_RATES = [1e-4, 1e-3, 5e-3, 1e-2]

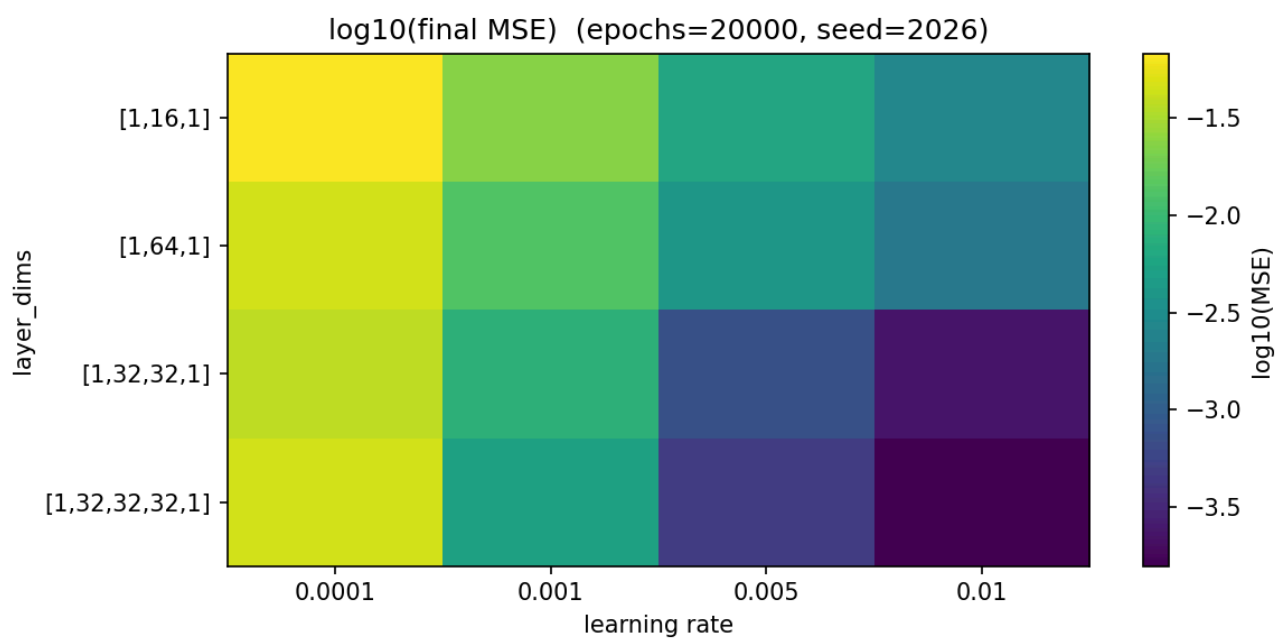
# 4 组网络结构
LAYER_DIMS_LIST = [
    [1, 16, 1],
    [1, 64, 1],
    [1, 32, 32, 1],
    [1, 32, 32, 32, 1],
]
```

得到收敛曲线图如下：

sin(x) regression (epochs=20000, seed=2026)



得到热力图如下:



2. 分类任务 (Classification)

2.1 数据集与预处理

本任务为 12 类图像分类。数据位于工程目录下的 `train/`，按类别分子文件夹存放（文件夹名为 1 至 12），每类若干 28x28 的 `.bmp` 二值图像。实现上在 `part1/classification.py` 的 `load_data` 中完成读取与预处理：按数字顺序整理类别编号，将每张图展平为 784 维特征向量，标签采用 12 维 one-hot 编码；随后在全数据集上打乱顺序，并按比例划分训练集与验证集，以验证集准确率衡量泛化能力。记输入维度 $D = 784$ 、类别数 $K = 12$ ，则训练与验证数据可表示为 $X_{train} \in \mathbb{R}^{N \times 784}$ 、 $Y_{train} \in \mathbb{R}^{N \times 12}$ 及对应的 X_{val} 、 Y_{val} 。

2.2 模型结构与损失函数

分类器采用多层全连接网络：隐层使用 `tanh` 激活，最后一层输出 logits，经 softmax 得到各类别概率。设 batch 大小为 N ，多类交叉熵损失写为

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^{12} y_{ic} \log p_{ic},$$

其中 p 为 softmax 概率， y 为 one-hot 标签。与平均交叉熵配套的 logits 梯度为 $\frac{\partial \mathcal{L}}{\partial z} = \frac{1}{N}(p - y)$ ，再向隐层逐层回传并在隐层处乘以 `tanh` 的导数。优化仍采用 SGD，与 Part1 回归部分一致。

2.3 程序实现说明

训练流程在 `part1/classification.py` 中实现为「前向计算损失 → 反向传播 → SGD 更新」。各模块分工如下表所示。

函数	作用
<code>set_seed</code>	固定随机种子，保证实验可复现。
<code>load_data</code>	读入图像、展平、one-hot、打乱并划分训练/验证集。
<code>init_params</code>	按 <code>layer_dims</code> 初始化权重与偏置。
<code>softmax</code>	将 logits 映射为概率向量。
<code>forward</code>	前向传播，缓存各层 <code>z[i]</code> 、 <code>a[i]</code> 及 <code>probs</code> 。
<code>cross_entropy_loss</code>	计算批量平均交叉熵。
<code>backward</code>	计算梯度并回传至各层参数。
<code>sgd_step</code>	按学习率更新参数。
<code>train</code>	封装训练循环（本文件中单次训练示例）。
<code>predict</code>	对输入样本输出预测类别（ <code>argmax</code> ）。

批量对比实验单独写在 `part1/classification_experiments.py` 中，便于固定同一数据划分与随机种子后扫描超参数。

2.4 实验设置

分类对比实验采用控制变量法：固定随机种子 `SEED=2026`、固定训练轮数 `EPOCHS=10000`（与 `classification_experiments.py` 中一致），分两组扫描。

第一组在固定网络结构 `[784, 64, 32, 12]` 下，比较 5 个学习率对收敛与验证准确率的影响；第二组在固定学习率 `lr=2e-3` 下，比较 4 种隐层宽度与深度配置。脚本中关键配置如下。

```
SEED = 2026
EPOCHS = 10000
BASE_LAYER_DIMS = [784, 64, 32, 12]
LR_LIST = [5e-4, 1e-3, 2e-3, 5e-3, 1e-2]
BASE_LR = 2e-3
ARCH_LIST = [
    [784, 32, 12],
    [784, 64, 12],
    [784, 64, 32, 12],
    [784, 128, 64, 12],
]
```

每组实验记录训练集交叉熵与验证集准确率随 epoch 的变化，并在训练结束后汇总最终验证准确率，结果同时写入 `part1/figures/classification_results.npz` 便于制表。

2.5 实验结果与分析

图 1 给出了固定结构下不同学习率的训练交叉熵与验证准确率曲线。可见随 epoch 增加，训练损失总体下降、验证准确率上升；学习率过小则前期下降偏慢，过大时曲线可能出现更明显的波动，需在收敛速度与稳定性之间折中。

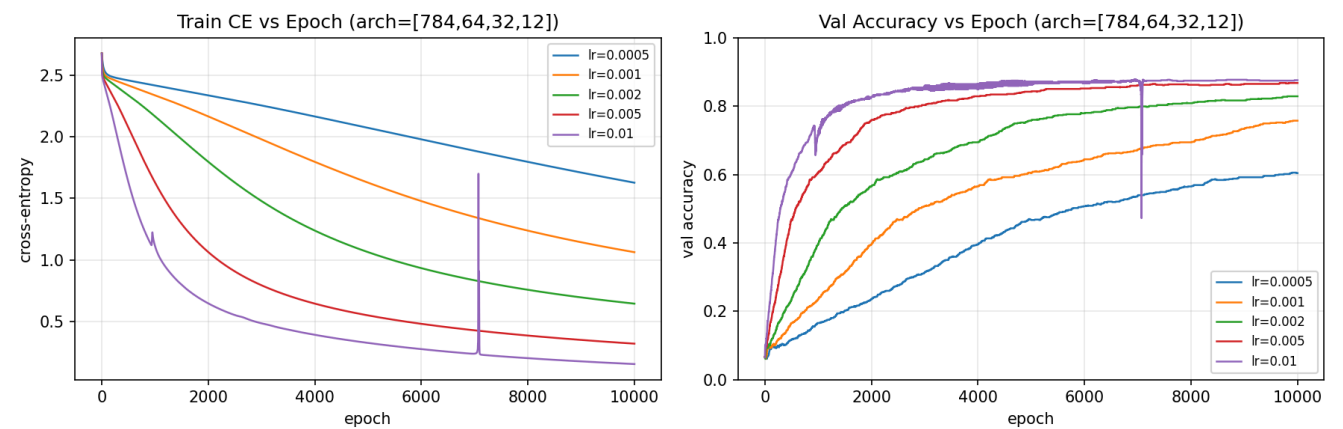


图 2 在固定 `lr=2e-3` 下比较了四种网络结构。更深或更宽的网络在本设置下通常能提供更强拟合能力，但也会带来训练时间增加，需在精度与开销之间权衡。

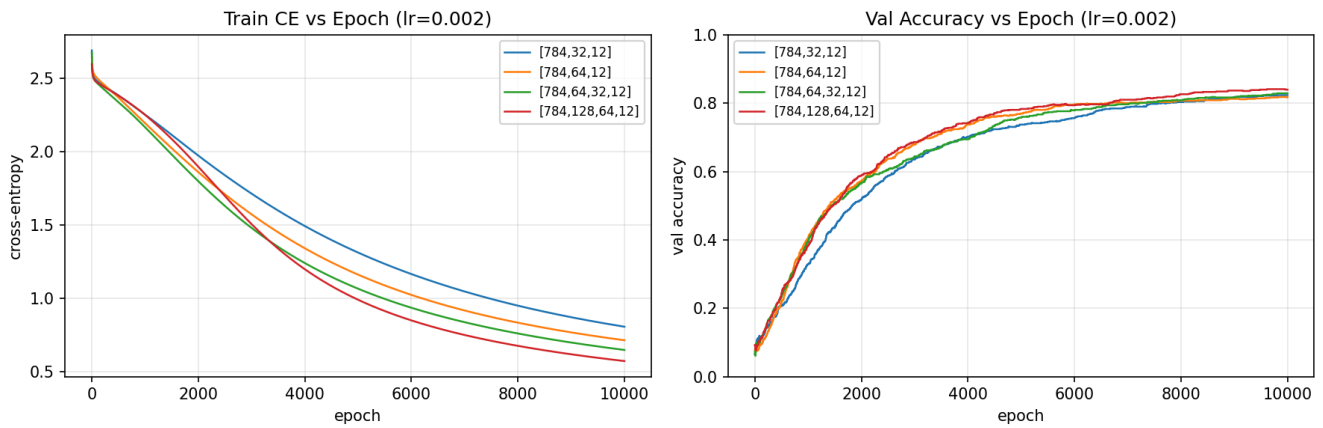
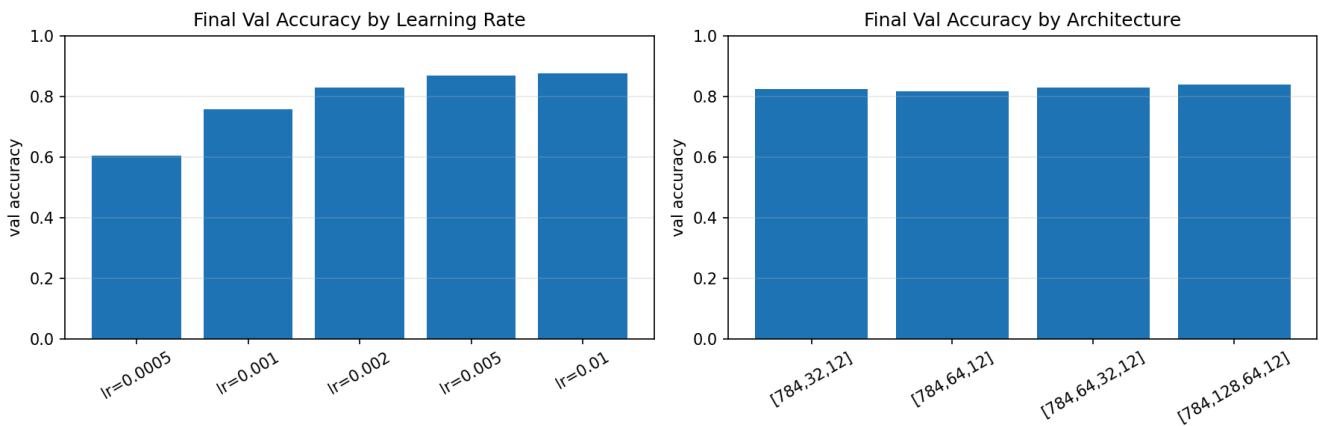


图 3 汇总了各组实验结束时的验证准确率。根据本次运行保存的 `classification_results.npz`，学习率扫描中验证集准确率最高约为 0.876（对应 $lr=1e-2$ ，结构为 `[784,64,32,12]`）；结构扫描中验证集准确率最高约为 0.840（对应 `[784,128,64,12]`，且 $lr=2e-3$ ）。两组实验的搜索空间不同，数值不宜直接横向等同为“全局最优”，更合理的做法是在验证集表现较好的区间内再做细化搜索。



Part 2

Part 2 要求使用 `NumPy` 自行实现卷积神经网络（未依赖 `PyTorch` 等框架），完成与 Part 1 分类任务相同的数据集上的 12 类手写汉字图像分类。

数据与 Part 1 分类部分一致：`train/` 下按类别 1 ~ 12 分子目录存放 28×28 二值图像。`load_data` 将每张图读入后展平为 784 维向量，标签为 12 维 one-hot；在全集上打乱后按比例划分训练集与验证集。训练与推理时通过 `images_flat_to_nchw` 将 $(N, 784)$ 转为 $(N, 1, 28, 28)$ ，以 **NCHW** 布局送入卷积。

2. 网络结构与实现思路

碍于手动实现的复杂性，本实验采用单层卷积 + ReLU + 最大池化 + 展平 + 全连接 + logits 的分类流水线：

$$\text{Conv2d} \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d} \rightarrow \text{Flatten} \rightarrow [\text{Dropout}] \rightarrow \text{Linear} \rightarrow z$$

超参设置： 3×3 卷积、`pad=1`、`stride=1` 保持空间尺寸为 28×28 ； 2×2 非重叠最大池化得到 14×14 ；卷积输出通道数为 `c_hidden`，全连接输入维度为 `c_hidden` $\times$ 14×14 ，输出维度为 12。损失为 batch 平均的 softmax + 交叉熵；对 logits 的梯度为 $\frac{1}{N}(p - y)$ ，再经 `cnn_backward` 逐层回传。

卷积实现：通过 `im2col` 将输入展成列矩阵，与展平后的卷积核矩阵相乘实现前向；反向中用 `col2im` 将 `dcol` 累加回输入梯度。`conv2d_forward` 中将 `out_col` reshape 为 $(C_{\{out\}}, out_h, out_w, N)$ 再 `transpose` 为 $(N, C_{\{out\}}, out_h, out_w)$ ，与 `im2col` 的列顺序（样本维变化最快）严格对齐，避免样本与空间位置错位。

正则化：全连接前可选 **Inverted Dropout** (`dropout_forward` / `dropout_backward`)；优化步中可选 **L2 权重衰减** (`sgd_step` 中 `grad += λw`)。验证集上若验证准确率若干轮不提升，可 **Early Stopping** 并返回历史最佳参数 (`train_cnn` 中 `patience`)。

测试集评测：`interview(eval_dir, params)` 按与训练集相同的目录结构读取 `eval_dir` 并打印测试准确率；`save_params` / `load_params` 将参数字典存为 `cnn_params.npz`，避免每次重新训练。

3. 主要函数说明

函数	作用
<code>set_seed</code>	固定 <code>numpy</code> 随机种子。
<code>images_flat_to_nchw</code>	将 (N,784) 转为 (N,1,28,28)。
<code>im2col</code> / <code>col2im</code>	卷积前向的 patch 展开与反向时梯度累加回输入。
<code>init_conv2d_weight</code> / <code>init_bias_conv</code>	He 初始化卷积核、零初始化卷积偏置。
<code>conv2d_forward</code> / <code>conv2d_backward</code>	二维卷积前向与反向 (含 pad 处理)。
<code>relu_forward</code> / <code>relu_backward</code>	ReLU 与掩码反向。
<code>maxpool2d_forward</code> / <code>maxpool2d_backward</code>	非重叠最大池化及梯度路由 (平局时均分)。
<code>dropout_forward</code> / <code>dropout_backward</code>	Inverted Dropout, 推理时关闭。
<code>flatten_forward</code> / <code>flatten_backward</code>	空间维展平与梯度 reshape。
<code>init_linear_weight</code> / <code>linear_forward</code> / <code>linear_backward</code>	全连接参数初始化、前向、反向。
<code>softmax</code> / <code>cross_entropy_loss</code> / <code>softmax_cross_entropy_grad_from_logits</code>	概率、损失及对 logits 的梯度。
<code>sgd_step</code>	SGD, 可选 <code>weight_decay</code> 。
<code>load_data</code>	读图、one-hot、打乱、划分 train/val。
<code>init_cnn_params</code>	初始化 <code>w_conv</code> , <code>b_conv</code> , <code>w_fc</code> , <code>b_fc</code> 与 <code>meta</code> (含 <code>drop_prob</code> 等)。
<code>cnn_forward</code> / <code>cnn_backward</code>	整条 CNN 前向与各层梯度汇总。
<code>predict_cnn</code> / <code>accuracy_from_onehot</code>	预测类别与准确率。
<code>save_params</code> / <code>load_params</code>	参数持久化与加载。
<code>interview</code>	对给定根目录 (与 <code>train/</code> 同结构) 批量评估准确率。
<code>train_cnn</code>	mini-batch SGD 训练; 可选 <code>return_history</code> 返回每 epoch 训练损失与验证准确率序列。

4. 对比实验设置

通过脚本，在固定随机种子与数据划分下做两组实验，关键配置如下：

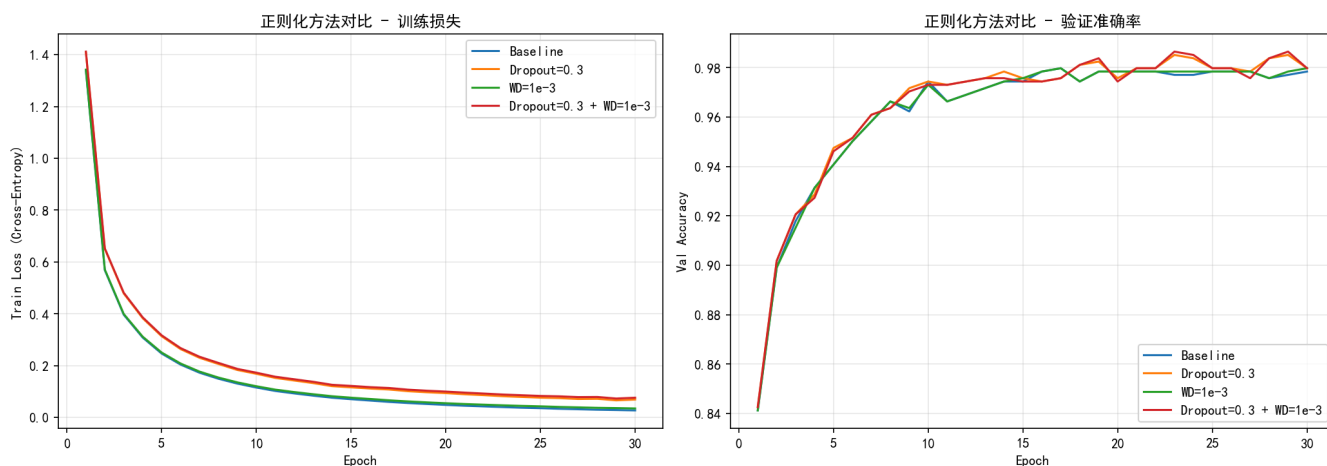
```
SEED = 2026
EPOCHS = 30
LR = 0.01
BATCH_SIZE = 16
# 第一组：固定 c_hidden=8，比较正则化
CONFIGS = {
    'Baseline': {'weight_decay': 0.0, 'drop_prob': 0.0},
    'Dropout=0.3': {'weight_decay': 0.0, 'drop_prob': 0.3},
    'WD=1e-3': {'weight_decay': 1e-3, 'drop_prob': 0.0},
    'Dropout=0.3 + WD=1e-3': {'weight_decay': 1e-3, 'drop_prob': 0.3},
}
# 第二组：固定 Dropout=0.3 与 WD=1e-3，比较卷积通道数
CHANNEL_CONFIGS = {'c_hidden=4': 4, 'c_hidden=8': 8, 'c_hidden=16': 16, 'c_hidden=32': 32}
```

每组记录每个 epoch 的平均训练交叉熵与整集验证准确率，并保存最终 train/val 准确率到

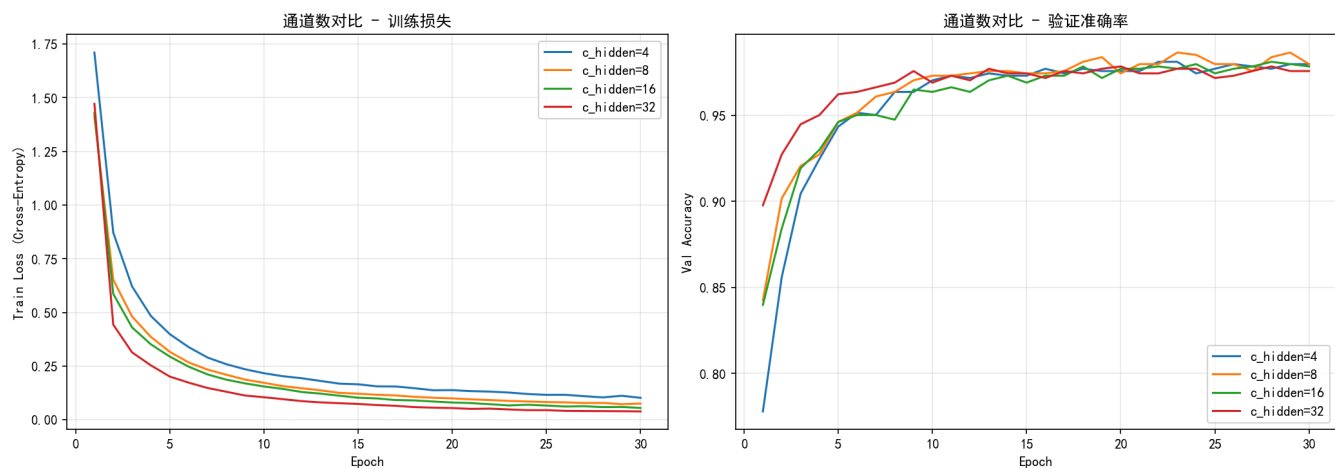
part2/figures/cnn_results.npz。

5. 实验结果与分析

下图为不同正则化策略下的训练损失与验证准确率曲线。无正则的 Baseline 与仅 L2 ($WD=1e-3$) 在本轮次未验证准确率相近 (约 0.980)；加入 Dropout 后验证准确率有所提升；Dropout 与 L2 同时启用时验证准确率最高，且训练/验证差距相对更小，说明组合正则有助于抑制轻微过拟合。



下图在固定 Dropout=0.3、 $WD=1e-3$ 下比较卷积通道数。通道从 4 增至 8 时验证准确率上升；继续增大到 16、32 时，在本数据量与划分下验证准确率未单调提升 ($c_hidden=32$ 时验证略降)，可能与大模型在有限数据上更易过拟合或验证集波动有关，需在更大验证集或交叉验证上进一步确认。



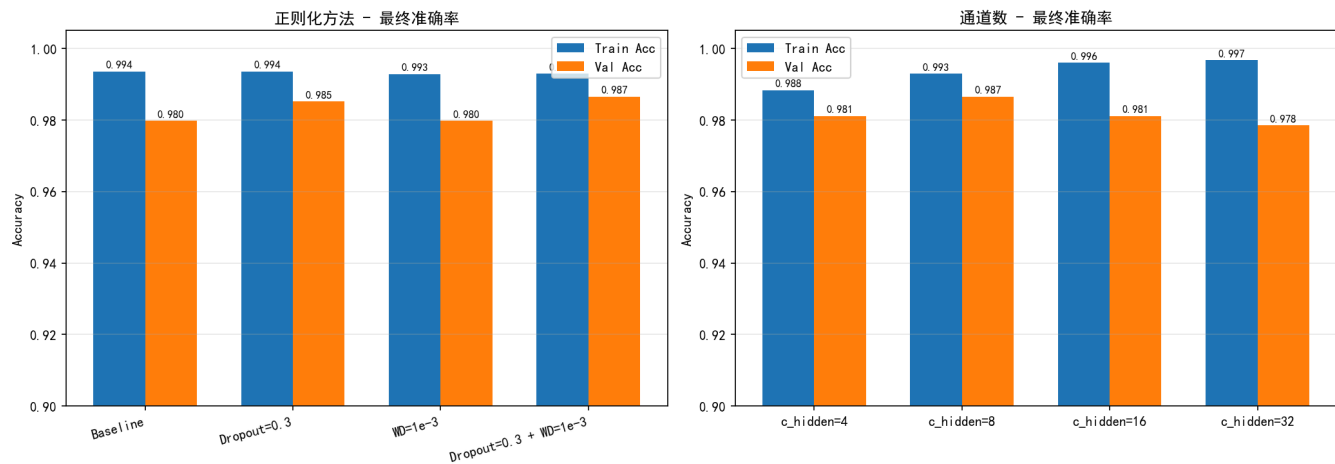
下面表格与柱状图汇总了各组实验结束时的训练集与验证集准确率。根据本次运行写入的 `cnn_results.npz`：

正则化组（c_hidden=8）最终准确率（与 `part2/figures/cnn_results.npz` 一致）

配置	Train Acc	Val Acc
Baseline	0.9936	0.9798
Dropout=0.3	0.9936	0.9852
WD=1e-3	0.9928	0.9798
Dropout=0.3 + WD=1e-3	0.9930	0.9865

通道数组（固定 Dropout+WD）最终准确率

配置	Train Acc	Val Acc
c_hidden=4	0.9883	0.9812
c_hidden=8	0.9930	0.9865
c_hidden=16	0.9960	0.9812
c_hidden=32	0.9967	0.9785



综上，在本实现与默认超参下，**适度正则（Dropout + 权重衰减）** 与 **适当通道数** 在验证集上取得了较好的折中，在本任务中可以取得较好的准确率。